THE QUESTIONS ON THE TABLE

Is there real demand for this? And what is it, exactly — a content-creation studio, or another AI playground?

The professionals' standard — and a studio, not a playground

The unit of work is a **node graph**, not a prompt — an inspectable pipeline where a tweak reruns one node, not the whole generation. A studio workload, a separate persona from any playground — **and it should ship as its own module, never merged into one.** Today it's a **single-user desktop app**: no auth, arbitrary-Python nodes, one GPU per instance.

HOW TEAMS RUN IT TODAY

- **A capable desktop under each desk** — $6–8k/seat, idle most of the day: four of the loop's five steps never touch the GPU. Cloud equivalent: a card each, **~$7,100/mo for ten**
- **One shared box** — a whole team queues on one unauthenticated process; any custom node runs *as the server*
- **Credit-metered SaaS** (Higgsfield, Krea, Runway) — $79–215/seat/mo team tiers; video by the second: a $35 plan ≈ **90 s of top-tier video**; data in the vendor's cloud

HOW FAST THIS MARKET MOVES — HIGGSFIELD ALONE

7 MONTHS	12 MONTHS	3 MONTHS
$1.3B → $5.4B	$20M → $700M	42×
valuation	annualized revenue	agentic-usage growth

Not the better product — **the proof of demand.** The commercial suites lead on go-to-market; ComfyUI leads the open-source side of the same category, and this repo is its go-to-market gap, closed.

ALREADY IN PRODUCTION ON COMFYUI

source: comfy.org

Amazon Studios · Apple · Autodesk · Netflix · Nike · Tencent · Ubisoft · Pixomondo · HP · Lucid

VFX & animation · advertising & creative studios · gaming · eCommerce & fashion

THE OPEN-SOURCE COUNTERWEIGHT

- ComfyUI: **$30M raise at a $500M valuation** (Apr 2026, Craft Ventures); 60,000+ community nodes
- Behind the first primarily-AI-generated Super Bowl ad; "**ComfyUI artist**" is a job title now
- Agents already speak it: Comfy Org ships an **official MCP server** — Claude Code, Codex & co. drive graphs as tool calls

THE GAP = THE PROJECT

The category's open-source leader has **no enterprise deployment story**: no SSO, no isolation, no shared-GPU economics, no per-user accounting.

Adopters: comfy.org · Higgsfield figures: Reuters / FT / PR Newswire, Aug 2026 · SaaS pricing: vendor team tiers, 2026

THE QUESTIONS ON THE TABLE

Can the expensive machine under each creator's desk be broken up — GPUs to the datacenter, frontend untouched?

Same ComfyUI, unchanged — the GPUs move to a pooled, scale-to-zero cluster

Keep the frontend identical — **stock ComfyUI at a pinned commit**, not a wrapper — and separate it from the backend: cards leave the desks and become one shared machine pool that turns off between jobs. Three pillars:

1 GPU cost as low as it goes

Pods **and nodes** scale to zero on queue depth; the queue converts one person's setup time into another's inference. Ten designers: **~$7,100/mo → $120–950/mo**. `showback` names whose GPU-seconds they were.

2 Shared storage, least friction

A checkpoint pulled once is on the volume for everyone — no 7 GB re-downloads per laptop. The 40–120 GB team library, LoRAs and outputs **outlive pods and the cluster itself**; crashed node ≠ lost afternoon.

3 The desktop experience, datacenter silicon

The loop below is **identical to the desktop** — except the run step lands on pooled L4s today and **L40S / A100 / H100 tiers** as jobs grow: a job declares what it needs, the queue places it. Nobody pays 80 GB prices for a 24 GB job.

THE DESIGNER'S LOOP — WHAT CHANGES FOR THE USER: ALMOST NOTHING

Load a template	→	See missing models	→	One-click load	→	Wire & fix the graph	→	Run — ~2 min
identical		Manager — identical		to shared storage, datacenter bandwidth		identical		the only GPU step

Four of five steps never touch a GPU — why ten designers need one pool and a queue, not ten cards. Savings widen with headcount: 33% at five people → 64% at thirty, plus caching that reruns only the node you changed.

SECURITY SEES

ComfyUI on loopback, **no Service, no Route**; namespace default-deny; cluster SSO; every grant audited.

FINANCE SEES

Per-user showback and monthly quota; three cost states, each one command (running / parked / torn down).

THE PLATFORM TEAM SEES

One namespace. Driver, nodes, control plane, TLS: the platform's pager — the OpenShift sell, verbatim.

Rates: g6.xlarge (L4 24 GB) at $0.976/h all-in — the repo's own cost model, reconciled in its README

THE QUESTIONS ON THE TABLE

What does this provide beyond the existing Omni–ComfyUI integration?

The production layer neither stock ComfyUI nor the bridge provides — built, tested, merged

~9,000 lines of Python and shell in `redhat-et/comfyui-on-openshift` : the queue, the isolation, the autoscaling, the accounting, and the failure handling you otherwise meet at 2am.

ARCHITECTURE — HUB AND SPOKE, WORKERS UNREACHABLE BY DESIGN

Browser
stock ComfyUI canvas

→

Route + SSO
oauth-proxy, namespace role

→

Gateway
FastAPI · showback · quota

→

Redis
queue + replayable progress log

→

GPU workers 0..N
ComfyUI on loopback · no Service, no Route

→

KEDA + machine pool
pods *and nodes* to zero; warm floor on a schedule

STATUS MERGED · CI GREEN

- On **master**, released and tagged; four CI jobs per PR + nightly GPU-image builds, booted as an arbitrary UID and scanned
- E2E suite runs the **real gateway and worker** on a laptop — no cluster, no GPU, ~1 min (`make test`); `make demo-local` goes further — the real pool **rendering on the laptop's own GPU**
- Shipped: per-user showback & quota, warm-floor scheduling, dead-worker reaping, per-user workspaces, default-deny policy
- Docs 01–13, incl. scaling to hundreds of users and a first-cluster-day checklist
- Ahead: VRAM-tier routing (L4 → L40S → H100) and pool-aware agent/MCP access — both on the written roadmap

RELATION TO `COMFYUI-VLLM-OMNI` (THE BRIDGE) COMPLEMENTARY

The bridge is the **right integration shape**: ComfyUI nodes that delegate a generation call to a vLLM Omni server, composing with everything else on the canvas.

What it leaves open — by its own README — is deployment: no auth, no failover, no progress streaming, one hand-started server per model. **The same list this platform closes.**

The bridge puts Omni on the canvas; this puts the canvas in production. Run here, it inherits SSO, network policy and the audit log for free.

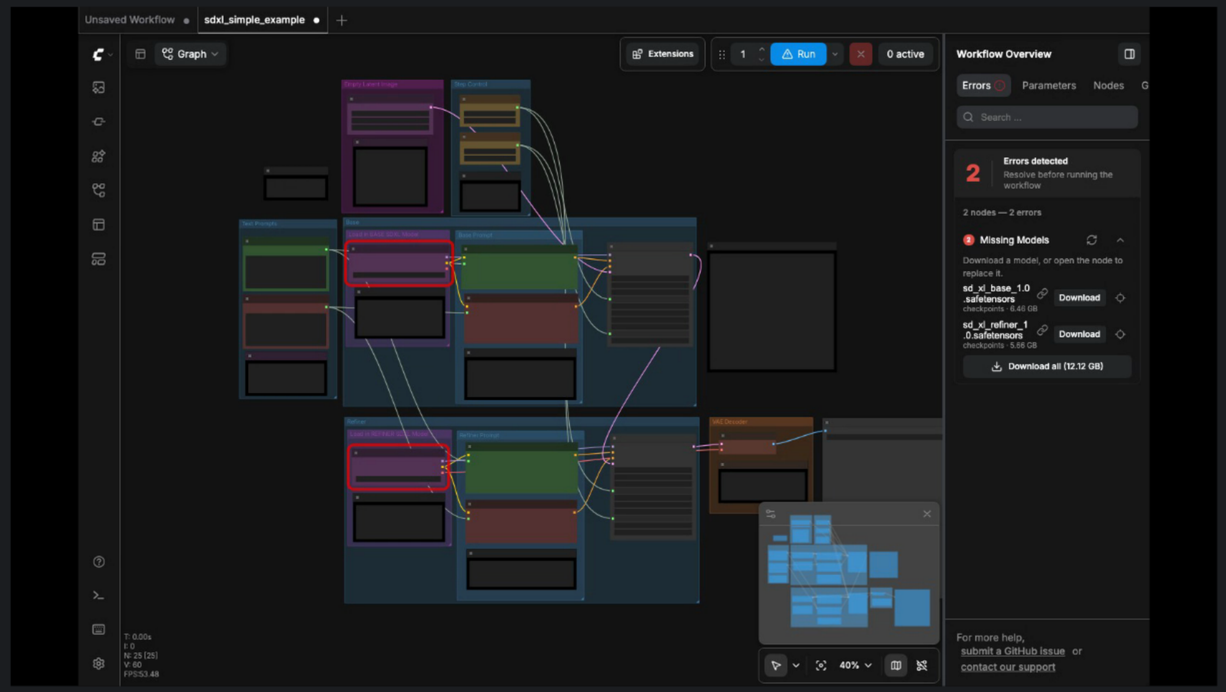
Scope, honestly: **the graph never leaves ComfyUI** — delegation carries coarse prompt→output steps, image-only today. How far that reaches is an open question the roadmap's engine spike exists to answer.

THE QUESTIONS ON THE TABLE

Can we see it work — the workflow designers actually have, not a mockup of it?

The designer's loop, live — only the silicon moved

`make demo-local` runs the real pool on the machine in front of you, zero dollars of cloud — and each worker serves **the stock ComfyUI canvas**, the exact frontend designers already run. Below, **the loop in one take** — the same single buttons as the desktop for the template and for resolving the graph; only where the models and the GPU live has changed.



stock canvas → SDXL template → two models **found · named · sized** → **Manager pulls 12.6 GB to the worker, server-side** (time-lapse) → errors clear → Run → live per-node progress → the render — one scripted take, real renders

THE DESKTOP LOOP, VERBATIM

- Load a template · wire the graph · run — **identical to the desktop**, same canvas, same nodes
- Missing models load **to the worker, not the laptop** — **ComfyUI-Manager**, server-side, onto the shared volume, pulled once for everyone
- Run streams progress per node; the render lands on shared storage

MANAGER IS THE HINGE

The add-on is what lets the desktop habit survive the move to a cloud backend: the designer keeps clicking the same buttons while **the models and the GPU live in the datacenter**. It ships in the worker image; the gateway stays the multi-user door.

HONESTLY NOT SHOWN

One laptop GPU time-shared — **pods simulated, silicon not**. SSO, KEDA and scale-to-zero need the cluster. The ladder: **$0 → $0 → ~$50** — `make test` , `make demo-local` , a live-cluster day.

demo-local-designer-loop.mp4 and demo-local-pool.mp4, beside this deck — the demo drives itself: every click is scripted, so the take reruns identically on any machine with a GPU

5 · THE ECONOMICS, IN ONE SENTENCE

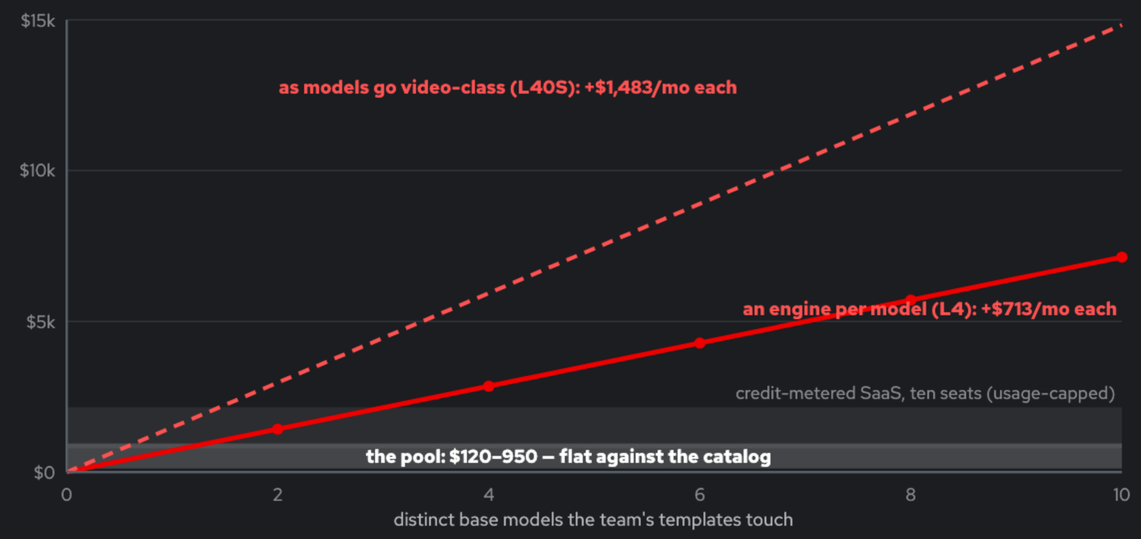
THE QUESTIONS ON THE TABLE

Why not simply serve every model behind an engine? What actually drives GPU cost for this workload?

Residency scales with catalog size. The pool scales with concurrency.

Everything on this page is that sentence, checked from a different side. The corollary: **the marginal model costs a machine on a serving tier, and disk on the pool.**

TEN PEOPLE, GROWING MODEL LIBRARY — MONTHLY GPU COST



WHY RESIDENCY ONLY GETS STEEPER

- **Catalogs only grow** — every template drags in a checkpoint (real libraries: 40–120 GB)
- **Two curves compound**: more models, on climbing cards — L4 $0.98 → L40S $2.03 → H100-class $7–12/hr
- **Quota compounds too**: N engines hold N GPUs of standing quota, the multi-day-ticket resource
- **Adapters don't save it**: LoRAs share a base; base checkpoints cannot share an engine

WHY THE POOL HOLDS FLAT

- **Demand is concurrency**: ten designers ≈ 4–24 GPU-h/day; six engines would supply 144 — idle, billed
- **Four of five loop steps never touch a GPU** — the pool wins below ~40% duty; savings widen with headcount (33% → 64%, five → thirty people)

WHERE THE THESIS FLIPS — BY DESIGN

When one model **concentrates** at sustained duty, residency wins *for that model* — the promotion rule, next slide. Two tiers, one design.

docs/13-vllm-omni.md — the arithmetic, the catalog curve, and the boundary of the bridge

THE QUESTIONS ON THE TABLE

Where does the strategic inference engine fit in this? And does any of it stray into the application layer?

The engine is the throughput tier here — and the application layer stays untouched

A serving engine holds **one model resident** behind an API — residency scales with catalog size; a design library is many models touched briefly — **the pool scales with concurrency**. And nothing here is an application layer: **ComfyUI is upstream's app at a pinned commit; this repo is deployment wiring.**

CORNER	WHO BUYS IT	THE TIER
Voice: TTS, speech, realtime agents — likely Omni's largest enterprise corner	Contact centers, IVR, accessibility — the classic Red Hat buyer	OMNI, RESIDENT
Generation called from application code — endpoint + SLA	Product teams; API-first customers	OMNI, RESIDENT
Designer image & video workflows — graphs, catalogs, 60k custom nodes	Studios, agencies, brand teams	THE POOL — THIS REPO
A hot model *inside* those workflows	Same studios, once showback shows concentration	OMNI BESIDE THE POOL, VIA THE BRIDGE
Omni-modal & robot-policy models	Robotics / agents programs	OMNI

The promotion rule: when `GET /api/showback` shows one model dominating GPU-seconds at sustained duty, stand up an Omni engine for it beside the pool, reached through the bridge nodes — same cluster, same SSO, same bill; **the marginal model costs a machine on a serving tier, and disk on the pool.** The bounded part, stated plainly: promotion carries a **coarse prompt→output call** — the graph itself never leaves ComfyUI, latent/ControlNet-level control does not fit an API, and the bridge is image-only today. How much of a real hot path fits that shape is an open question; showback answers it with data, per team.

THE ASKS

- **PM read** on the persona (30 min): a content-creation *studio* workload — is this a corner we stand in?
- **Customer-scenario session**: map the API-first scenarios against the two tiers
- **Demo budget**: a full live-cluster demo day is ~$25–50 at the repo's own rates
- **Name the owner** of the "does this have a home" decision

ZERO-COST DUE DILIGENCE, TODAY

`make test` runs the real gateway and worker on a laptop in about a minute — no cluster, no GPU, no AWS account — and `make demo-local` renders through the same pool on the laptop's own GPU (slide 4). The README's comparison table and `docs/13-vllm-omni.md` are the homework in written form; a workflow blog post series is the cheapest proof after that.

docs/13-vllm-omni.md — the full cost arithmetic and the composition rule